

Руководство по программированию подводного робота Кит

Что нужно для начала работы с Китом

Необходимо:

- Собранный экземпляр [подводного робота](#), с заряженной батареей.
 - Компьютер под управлением ОС Windows или Linux. Можно также использовать MacOS, но официальной поддержки ее в данном руководстве нет. Обязательно нужна поддержка Wi-Fi, диапазона 2,4 ГГц.
 - Доступ к сети Интернет или предварительно скачанные и установленные утилиты, примеры кода и модули Python.
 - Версия [Python](#) 3.14+ на компьютере для запуска скриптов взаимодействия с роботом.
 - Владение ЯП Python, базовые знания командной строки Linux.
 - В случае использования ОС Windows, скачайте и установите в систему следующие утилиты:
 - WinSCP, можно скачать с <https://winscp.net>
 - PuTTY, можно скачать с <https://putty.org>
- В случае использования ОС Linux, нужные утилиты уже будут установлены в системе.

Желательно:

- Использование одной из доступных IDE для работы с Python, например [Visual Studio Code](#)

Подключение к роботу

Соединение с платой управления роботом на Raspberry Pi осуществляется через Wi-Fi адаптер, расположенный на надводном бую. Он служит как маршрутизатор сетевых пакетов и DHCP сервер.

Настройка подключения:

- Включите питание робота. Подождите, пока он полностью загрузится (это может занять 1-2 минуты).
- Убедитесь, что на компьютере включен Wi-Fi адаптер (в некоторых ноутбуках он может выключаться одной из Fn кнопок на клавиатуре). В ОС Linux может потребоваться использование утилиты `rkill` для разблокировки беспроводных адаптеров.
- Осуществите поиск доступных сетей, нужная сеть будет называться `oceanikaXXX`, где XXX - цифры, указанные на шильдике буя. Подключитесь к ней, указав пароль 12345678. Внимание: эта сеть не имеет доступа к Интернет!
- В случае успешного подключения проверьте связь с платой управления, послав эхо-запрос (`ping`) из командной строки:

```
ping 192.168.88.155
PING 192.168.88.155 (192.168.88.155) 56(84) bytes of data.
64 bytes from 192.168.88.155: icmp_seq=1 ttl=64 time=6.0 ms
```

Внутри робота, плата Raspberry Pi 4B работает под управлением ОС Linux, Ubuntu 20.04. Поэтому для копирования файлов в файловую систему робота и обратно необходимо использовать протокол SFTP, а для подключения к консоли (удаленному терминалу) протокол SSH.

Параметры подключения следующие:

Протокол	IP адрес	Порт	Логин	Пароль
SFTP/SSH	192.168.88.155	22	pi	oceanika

В ОС Windows для работы с протоколом SFTP рекомендуется использовать утилиту WinSCP, а для SSH – использовать PuTTY.

В ОС Linux эту роль играют утилиты `scp` и `ssh` соответственно.

Например, команда:

```
scp test.py \
pi@192.168.88.155:/usr/local/drone_ros/src/drone/scripts/examples/
```

скопирует файл `test.py` из текущей папки в удаленную папку (директорию) в файловой системе робота.

Команда:

```
ssh pi@192.168.88.155
```

подключится к удаленному терминалу и запросит пароль.

Для выполнения некоторых команд в удаленном терминале робота, требующих привилегии администратора (суперпользователя), необходимо будет использовать команду `sudo`. В этом случае система запросит пароль пользователя `pi` еще раз.

Создание нового скрипта управления

В качестве примера разработки, возьмем один из имеющихся скриптов и модифицируем его под новую задачу.

Допустим, она состоит в том, чтобы двигаться вперед и назад в темноте (с выключенными огнями), а разворот делать с включенными на 80% мощности фонарями.

Выполните следующие шаги:

- Создайте локальную папку, в которой будет лежать код скрипта на этапе разработки и отладки. В ней же можно создать новый пустой Python проект VS Code.
- Скопируйте (с помощью WinSCP или `scp`) файл `/usr/local/drone_ros/src/drone/scripts/examples/forward.py` в локальную папку и переименуйте его в `test.py`.

Вот этот файл, с добавленными номерами строк для удобства:

```
1. #!/usr/bin/env python3
2.
3. from user.library import DroneLibrary
4. import time
5.
6. if __name__ == "__main__":
7.     speed = 50
8.
9.     drone = DroneLibrary()
```

```

10.     drone.start(False)
11.
12.     drone.set_course(drone.get_course())
13.     drone.set_depth(drone.get_depth())
14.     drone.set_offline_mode()
15.
16.     drone.set_speed(0)
17.     for i in range(0, 10):
18.         drone.set_speed(speed)
19.         print(f'depth: {drone.get_depth()}')
20.         time.sleep(5)
21.         drone.set_speed(0)
22.         print(f'depth: {drone.get_depth()}')
23.         drone.change_course(180)
24.         time.sleep(3)
25.
26.     drone.stop()

```

Строка 1: так называемый shebang, стандартный способ в Unix указать требуемый интерпретатор для скрипта.

Строки 3, 4: импорт используемых модулей Python, **DroneLibrary** представляет собой библиотеку управления подводным роботом.

Строка 6: точка входа в скрипт.

Строки 9, 10: создание экземпляра класса и запуск основного цикла работы библиотеки (в отдельном треде). Опциональный аргумент метода `start()` позволяет полностью перехватить управление роботом (`True`, значение по умолчанию) или оставить возможность параллельного управления с джойстика (`False`).

Строки 12, 13: получают и фиксируют текущий курс и глубину. При правильно откалиброванном роботе внешне ничего не должно измениться.

Строка 14: переход в автономный режим, робот не будет реагировать на джойстик.

Строка 16: выключает моторы робота. С учетом инерции он остановится не сразу!

Строки 18-21: включает моторы робота на 5 секунд на половине скорости вперед.

Строки 23, 24: разворачивает робота на 180° вокруг вертикальной оси (рыскание)

Строка 26: остановка главного цикла библиотеки, возврат управления джойстику.

Внесите модификации в файл:

- После строки 16 добавьте `drone.set_headlight(0)`
- Перед строкой 23 добавьте `drone.set_headlight(80)`
- После строки 24 добавьте `drone.set_headlight(0)`

Скопируйте файл `test.py` обратно

в `/usr/local/drone_ros/src/drone/scripts/examples/test.py`

Внесите изменения в файл `/usr/local/drone_ros/src/drone/CMakeLists.txt`, добавив `scripts/examples/test.py` к списку `catkin_install_python`.

Запуск и отладка

Перед запуском, перейдите в директорию `/usr/local/drone_ros/` и соберите код:

```

cd /usr/local/drone_ros/
sudo systemctl stop oceanika.service
./build.sh

```

```
sudo systemctl start oceanika.service
```

Выполнять сборку необходимо только при внесении исправлений в файл `/usr/local/drone_ros/src/drone/CMakeLists.txt`, файлы `*.msg`, `*.srv`, `*.action`, или в исходный код частей проекта на C++.

При дальнейших изменениях кода только в `py` файлах, в пересборках необходимости нет!

Для запуска примера кода необходимо выполнить:

```
cd /usr/local/drone_ros/  
./example.sh test.py
```

Логи, использующие методы `print()` или `rospy.loginfo()`, `rospy.logerr()` будут выведены на экран во время выполнения скрипта.

Также, могут оказаться полезными следующие утилиты ROS:

```
# выводит список всех активных топиков  
rostopic list  
# выводит сообщения топика /control/hand в реальном времени  
rostopic echo /control/hand  
# показывает структуру сообщения /control/orientation  
rosmmsg show /control/orientation
```

Управление роботом через ROS Bridge

Роботом можно управлять кодом, запускаемым на компьютере, передавая сообщения в `топики /control` и получая сообщения от сенсоров через программный шлюз ROS Bridge, использующий протокол WebSocket.

URL для подключения к ROS Bridge: `ws://192.168.88.155:9090/`

Пример такого кода с использованием библиотеки `roslibpy` приведен в https://github.com/OceanikaProject/drone_vision

Полезные ссылки

1. [API Reference — roslibpy 2.1.0 documentation](#)
2. [ROS Noetic Ninjemys](#)
3. [Введение в ROS](#)

Приложение 1. Описание методов в DroneLibrary

Публичные API-методы класса DroneLibrary библиотеки `user.library`:

```
change_course(angle)  
change_depth(depth)  
change_pitch(pitch)  
change_roll(roll)  
change_speed(speed)  
get_course()  
get_depth()  
get_pitch()  
get_roll()  
set_control(state)
```

```
set_course(angle)
set_depth(depth)
set_hand(value)
set_headlight(value)
set_laser(value)
set_offline_mode()
set_online_mode()
set_payload(value)
set_pitch(pitch)
set_roll(roll)
set_speed(speed)
start(takecontrol=True)
stop()
```

Приложение 2. Список топиков проекта drone_ros

← Whale programming guide 

```
/control/motor/filter/auto
/control/motor/filter/manual
/control/motor/limit
/control/motors
/control/orientation
/control/override
/control/payload
```

Топики камеры:

```
/raspicam_node/camera_info
/raspicam_node/image
/raspicam_node/image/compressed
/raspicam_node/motion_vectors
```

Топики сенсоров:

```
/sensor/depth
/sensor/euler
/sensor/imu
/sensor/voltage
```

Приложение 2. Глоссарий

Крен (roll), рысканье (yaw), тангаж (pitch)



Установите Google Документы!

Вносите изменения, оставляйте комментарии и редактируйте файлы одновременно с другими пользователями.

НЕТ

УСТАНОВИТЬ ПРИЛОЖЕНИЕ